

# Comprehensive Documentation Report:

## PFCOMPAS-v1.0.0

### Adaptive MATLAB Framework for Phase-Field Modeling of Fracture

Pranjal Saxena      Chandra Shekhar Upadhyay

## Contents

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Motivation</b>                                                       | <b>4</b>  |
| <b>2</b> | <b>Mathematical Formulation</b>                                         | <b>5</b>  |
| 2.1      | Kinematics . . . . .                                                    | 5         |
| 2.2      | Elastic Constitutive Response . . . . .                                 | 6         |
| 2.3      | Phase-Field Representation of Fracture . . . . .                        | 6         |
| 2.4      | Degradation of the Elastic Energy . . . . .                             | 6         |
| 2.5      | Tension-Compression Energy Splits . . . . .                             | 7         |
| 2.5.1    | Isotropic Degradation . . . . .                                         | 7         |
| 2.5.2    | Amor Volumetric-Deviatoric Split . . . . .                              | 7         |
| 2.5.3    | Miehe Spectral Split . . . . .                                          | 8         |
| 2.6      | Generic Regularized Fracture Surface Energy . . . . .                   | 8         |
| 2.7      | Total Potential Energy . . . . .                                        | 8         |
| 2.8      | Governing Mechanical Equilibrium Equation . . . . .                     | 8         |
| 2.9      | Generic Phase-Field Evolution Equation . . . . .                        | 9         |
| 2.10     | Thermodynamic Irreversibility . . . . .                                 | 9         |
| 2.11     | Boundary and Initial Conditions . . . . .                               | 9         |
| 2.12     | Specific PFCOMPAS-v1.0.0 Model Choices . . . . .                        | 10        |
| 2.13     | Variational Structure of the Numerical Solution . . . . .               | 10        |
| <b>3</b> | <b>PFCOMPAS-v1.0.0: Solver Architecture and Computational Algorithm</b> | <b>10</b> |
| 3.1      | Overview . . . . .                                                      | 10        |
| 3.2      | Incremental Loading and Accepted States . . . . .                       | 11        |
| 3.3      | Staggered Solution Strategy . . . . .                                   | 11        |
| 3.4      | Nonlinear Mechanical Equilibrium . . . . .                              | 11        |
| 3.4.1    | Mechanical Newton Convergence . . . . .                                 | 11        |
| 3.5      | History-Field Update . . . . .                                          | 12        |
| 3.6      | Phase-Field Newton Problem & Active-Set Treatment . . . . .             | 12        |
| 3.6.1    | Phase-Field Newton Convergence . . . . .                                | 12        |
| 3.7      | Staggered Convergence . . . . .                                         | 12        |
| 3.8      | Adaptive Load Stepping . . . . .                                        | 13        |
| 3.9      | Load-Step Rejection and Rollback . . . . .                              | 13        |

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| <b>4</b> | <b>Advanced Constitutive Algorithms</b>                                  | <b>14</b> |
| 4.1      | Constitutive Degradation and Cohesive Zone Modeling . . . . .            | 14        |
| 4.1.1    | The Role of Residual Stiffness . . . . .                                 | 14        |
| 4.1.2    | Beyond Quadratic: Delaying Crack Nucleation . . . . .                    | 15        |
| 4.1.3    | Algorithmic Necessity of Exact Analytical Derivatives . . . . .          | 15        |
| 4.2      | Tension-Compression Energy Splits and Asymmetry . . . . .                | 15        |
| 4.2.1    | The Amor (Volumetric-Deviatoric) Split . . . . .                         | 16        |
| 4.2.2    | The Miehe Spectral Split . . . . .                                       | 16        |
| 4.2.3    | Algorithmic Regularization of the Macaulay Bracket . . . . .             | 16        |
| 4.2.4    | Hyperbolic Smoothing for $C^1$ -Continuity . . . . .                     | 16        |
| 4.2.5    | Eigenvalue Derivatives for the Spectral Tangent . . . . .                | 17        |
| 4.2.6    | Dynamic Scale Invariance . . . . .                                       | 17        |
| 4.2.7    | Algorithmic Extraction of the Amor Consistent Tangent . . . . .          | 17        |
| 4.2.8    | Volumetric Tangent Extraction for Passive Compression . . . . .          | 17        |
| 4.2.9    | Kinematic Perturbation via Voigt Basis Tensors . . . . .                 | 18        |
| 4.2.10   | Kinematic Basis Tensors and Engineering Shear Mapping . . . . .          | 18        |
| 4.2.11   | Fréchet Derivatives and the Spectral Consistent Tangent . . . . .        | 18        |
| 4.2.12   | Derivatives of Isotropic Tensor Functions . . . . .                      | 18        |
| 4.2.13   | Divided Differences and Eigenvector Rotation . . . . .                   | 18        |
| 4.2.14   | L'Hôpital's Rule for Repeated Eigenvalues . . . . .                      | 19        |
| <b>5</b> | <b>Spatial Discretization and Finite Elements</b>                        | <b>19</b> |
| 5.1      | Motivation for a Unified Mesh Database . . . . .                         | 19        |
| 5.1.1    | Explicit Degree of Freedom (DOF) Mapping . . . . .                       | 19        |
| 5.1.2    | High-Performance Sparse Matrix Pre-Allocation . . . . .                  | 19        |
| 5.1.3    | Parallel Element Evaluation and Vectorized Assembly . . . . .            | 20        |
| 5.1.4    | Algorithmic Requirements for Verfürth Estimators . . . . .               | 20        |
| 5.1.5    | Hash-Map Based Interface Detection . . . . .                             | 20        |
| 5.1.6    | Data Structure for Jump Residuals . . . . .                              | 20        |
| 5.1.7    | Geometric Boundary Classification . . . . .                              | 21        |
| 5.2      | Element Interpolation and Lagrange Basis Functions . . . . .             | 21        |
| 5.2.1    | The Role of Shape Functions and Reference Coordinates . . . . .          | 21        |
| 5.2.2    | The Necessity of Higher-Order Elements in Phase-Field Fracture . . . . . | 21        |
| 5.3      | Kinematics and the Strain-Displacement Operator . . . . .                | 21        |
| 5.3.1    | Voigt Notation and Engineering Shear Strains . . . . .                   | 22        |
| 5.3.2    | Work-Conjugate Stress Mapping . . . . .                                  | 22        |
| 5.3.3    | Pristine Isotropic Elasticity and Energy Equivalence . . . . .           | 22        |
| 5.3.4    | Plane Strain Enforcement in 2D Phase-Field . . . . .                     | 22        |
| 5.3.5    | Thermodynamic Consistency of Shear Moduli . . . . .                      | 23        |
| 5.4      | Numerical Integration and Quadrature Schemes . . . . .                   | 23        |
| 5.4.1    | The Necessity of High-Order Quadrature in Phase-Field Modeling . . . . . | 23        |
| 5.4.2    | Multidimensional Tensor Products and Normalization . . . . .             | 23        |
| 5.4.3    | 1D Quadrature for Boundary and Interface Integration . . . . .           | 23        |
| 5.5      | Phase-Field Initialization and Crack Regularization . . . . .            | 23        |
| 5.5.1    | The Homogeneous Screened Poisson Equation . . . . .                      | 24        |
| 5.5.2    | Matrix Partitioning and Linear Solve . . . . .                           | 24        |
| 5.5.3    | Numerical Bounding and Symmetry Enforcement . . . . .                    | 24        |
| 5.5.4    | Element-Level Weak Form and Isoparametric Mapping . . . . .              | 24        |

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| 5.5.5    | Numerical Integration via Jacobian Transformation . . . . .         | 25        |
| <b>6</b> | <b>Main Execution Script and Global Architecture</b>                | <b>25</b> |
| 6.1      | Initialization and Solver Dispatch . . . . .                        | 25        |
| 6.2      | High-Performance Mechanical Assembly . . . . .                      | 25        |
| 6.3      | Thread-Safe History Field Extraction . . . . .                      | 26        |
| 6.4      | Vectorized Phase-Field Assembly and Active-Set Projection . . . . . | 26        |
| 6.5      | Global Energy Post-Processing . . . . .                             | 27        |
| 6.6      | Boundary Traction Integration and Reaction Forces . . . . .         | 27        |
| <b>7</b> | <b>A Posteriori Error Estimation and Diagnostics</b>                | <b>27</b> |
| 7.1      | Mechanical Displacement Error Estimator ( $\eta_u$ ) . . . . .      | 28        |
| 7.1.1    | Volumetric Residual Simplification . . . . .                        | 28        |
| 7.1.2    | Traction Jump Residuals and Dirichlet Masking . . . . .             | 28        |
| 7.2      | Phase-Field Damage Error Estimator ( $\eta_\phi$ ) . . . . .        | 28        |
| 7.2.1    | Internal Diffusion and Edge Flux Jumps . . . . .                    | 29        |
| 7.2.2    | Global Mixed Indicator ( $\eta_{\text{mix}}$ ) . . . . .            | 29        |
| <b>8</b> | <b>High-Fidelity Visualization and VTK Data Export</b>              | <b>29</b> |
| 8.1      | Timestep-Local Memory Architecture . . . . .                        | 30        |
| 8.2      | Data Segregation: Kinematics vs. Mechanics . . . . .                | 30        |
| 8.3      | C++ Indexing Parity . . . . .                                       | 30        |
| 8.3.1    | Topological Translation and VTK Cell Dictionaries . . . . .         | 30        |
| 8.4      | Dynamic Field Parsing and Array Validation . . . . .                | 31        |
| 8.5      | Temporal Collections and Pseudo-Time Tracking (.pvd) . . . . .      | 31        |
| 8.5.1    | HPC File I/O Safety Protocol . . . . .                              | 31        |
| 8.5.2    | Strict Topological Validation . . . . .                             | 32        |
| 8.5.3    | Dynamic Timestep Appending and Dataset Portability . . . . .        | 32        |
| 8.5.4    | XML Finalization and Pipeline Closure . . . . .                     | 32        |

# 1 Motivation

Phase-field fracture simulations combine a variational fracture formulation with nonlinear finite-element mechanics, making their implementation substantially more involved than that of a conventional finite-element elasticity solver. In practice, a user must simultaneously deal with the phase-field regularization length  $\ell_0$ , nonlinear constitutive degradation, tension-compression splits, irreversible crack evolution, nonlinear staggered solution procedures, load-step control, and the numerical resolution of the diffuse crack. PFCOMPAS-v1.0.0 was developed to bring these components together within a single, transparent computational framework.

The primary purpose of this manual is therefore practical: to document how the PFCOMPAS-v1.0.0 solver is formulated, organized, and executed so that a researcher can understand the complete computational workflow, reproduce simulations, modify individual solver components, and diagnose numerical difficulties. The manual is intended to serve as both a user guide to the existing implementation and a technical reference for further development of the code.

The underlying fracture formulation is based on the variational phase-field approach, in which a sharp crack is represented by a continuous phase field  $\phi \in [0, 1]$ . This avoids explicit crack tracking and remeshing during crack propagation, but introduces a coupled nonlinear problem in the displacement field and the phase field. In the AT2 formulation adopted by PFCOMPAS-v1.0.0, the phase field is coupled to the elastic response through a degradation function  $g(\phi)$ . For fracture problems in which tensile and compressive states must be distinguished, the elastic energy can additionally be decomposed into active and passive contributions.

PFCOMPAS-v1.0.0 provides several constitutive choices for this purpose, including the isotropic, Amor volumetric-deviatoric, and Miehe spectral tension-compression splits. These formulations differ both physically and numerically. In particular, the spectral formulation involves principal-strain decomposition and a non-smooth Macaulay-bracket operation, requiring appropriate regularization and consistent tangent evaluation for robust Newton-Raphson solution. The code therefore documents not only the governing constitutive equations, but also the numerical procedures required to evaluate their residuals and consistent tangents.

The manual also documents the complete solution process used to advance a simulation. Phase-field fracture can exhibit long nearly elastic portions followed by rapid crack initiation and propagation, making a fixed load increment inefficient and, in some cases, inadequate. PFCOMPAS-v1.0.0 therefore employs an adaptive load-stepping procedure that adjusts the increment according to the observed difficulty of the nonlinear solution. Failed increments are rejected and the previously accepted state is restored before a smaller increment is attempted. Understanding this accepted-state, trial-state, and rollback mechanism is essential for interpreting solver behavior and for modifying the load-control strategy.

A second practical requirement is distinguishing convergence of the nonlinear algebraic equations from accuracy of the finite-element discretization. Newton and staggered residual tolerances indicate whether the current discrete problem has been solved to the specified tolerances, but they do not by themselves establish whether the mesh adequately resolves the solution or the phase-field length scale  $\ell_0$ . PFCOMPAS-v1.0.0 therefore includes Verf’urth-type a posteriori error estimators for both the displacement and phase-field equations. These indicators provide localized and global measures that can be used

to assess mesh adequacy, investigate convergence, and identify regions where the spatial discretization error is concentrated.

From a software perspective, the manual explains how these physical and numerical components are translated into an efficient MATLAB implementation. The code separates mesh preprocessing, numerical integration, element-level calculations, global assembly, nonlinear solution, diagnostics, and output. Precomputed degree-of-freedom mappings and sparse assembly patterns, together with parallel element evaluation and vectorized global assembly, are used to reduce the computational cost of repeatedly assembling the coupled finite-element systems.

Accordingly, the purpose of this manual is not only to describe the mathematical model, but to provide a practical map of the entire PFCOMPAS-v1.0.0 codebase. It explains where each part of the formulation is implemented, how data move between the different solver stages, which numerical criteria determine convergence or step rejection, and which diagnostic quantities can be used to assess the reliability of a simulation. A reader should be able to use the manual to configure a problem, understand the resulting computational sequence, identify the relevant routines when modifying the solver, and interpret the principal numerical and physical diagnostics produced by the code.

## 2 Mathematical Formulation

The PFCOMPAS-v1.0.0 framework is formulated as a variational phase-field model for brittle fracture within the small-strain, quasi-static setting. The sharp crack is replaced by a regularized scalar phase field, denoted by  $\phi$ , which continuously describes the transition from the intact material to the fully damaged state. The formulation follows the variational fracture framework of Francfort-Marigo and its phase-field regularization, with the AT2 crack-density functional adopted as the specific fracture model.

The primary unknowns are the vector displacement field  $\mathbf{u}$  and the scalar phase-field variable  $\phi$ . The mechanical and phase-field problems are coupled through the degradation of the tensile elastic energy. Thermodynamic irreversibility is represented through a history variable  $\mathcal{H}$  and, at the discrete level, through a lower-bound constraint on the phase field.

### 2.1 Kinematics

The displacement field is defined over the body  $\Omega$  as

$$\mathbf{u} : \Omega \rightarrow \mathbb{R}^d \quad (1)$$

where  $d = 2$  for the present two-dimensional implementation. Under the infinitesimal-strain assumption, the symmetric strain tensor is defined as:

$$\boldsymbol{\varepsilon}(\mathbf{u}) = \frac{1}{2} [\nabla \mathbf{u} + (\nabla \mathbf{u})^T] \quad (2)$$

For finite-element implementation, the strain tensor is represented in Voigt notation. PFCOMPAS-v1.0.0 specifically utilizes engineering shear strains ( $\gamma_{xy} = 2\varepsilon_{xy}$ ). In two dimensions, the strain vector is:

$$\boldsymbol{\varepsilon}_V = \begin{bmatrix} \varepsilon_{xx} \\ \varepsilon_{yy} \\ \gamma_{xy} \end{bmatrix} \quad (3)$$

The corresponding stress vector is written as:

$$\boldsymbol{\sigma}_V = \begin{bmatrix} \sigma_{xx} \\ \sigma_{yy} \\ \sigma_{xy} \end{bmatrix} \quad (4)$$

With this convention, the tensorial and Voigt representations of the internal virtual work are mathematically identical:

$$\boldsymbol{\sigma} : \delta \boldsymbol{\varepsilon} = \boldsymbol{\sigma}_V^T \delta \boldsymbol{\varepsilon}_V \quad (5)$$

The discrete finite-element strain-displacement relation maps the nodal displacements  $\mathbf{u}_e$  to the integration points via the spatial derivative matrix  $\mathbf{B}$ :

$$\boldsymbol{\varepsilon}_V = \mathbf{B} \mathbf{u}_e \quad (6)$$

## 2.2 Elastic Constitutive Response

Prior to degradation, the material is assumed to be isotropic linear elastic with Young's modulus  $E$  and Poisson's ratio  $\nu$ . The intact elastic energy density is written in generic tensorial form as:

$$\psi_e(\boldsymbol{\varepsilon}) = \frac{1}{2} \boldsymbol{\varepsilon} : \mathbb{C}_0 : \boldsymbol{\varepsilon} \quad (7)$$

where  $\mathbb{C}_0$  is the fourth-order elastic stiffness tensor.

For the present two-dimensional implementation, a plane-strain constitutive matrix is enforced:

$$\mathbf{D}_{ps} = \frac{E}{(1+\nu)(1-2\nu)} \begin{bmatrix} 1-\nu & \nu & 0 \\ \nu & 1-\nu & 0 \\ 0 & 0 & \frac{1-2\nu}{2} \end{bmatrix} \quad (8)$$

yielding the constitutive relation  $\boldsymbol{\sigma}_V = \mathbf{D}_{ps} \boldsymbol{\varepsilon}_V$ . The use of engineering shear strains is consistent with the shear modulus appearing directly in the shear entry of the constitutive matrix, perfectly preserving the energy equivalence  $\frac{1}{2} \boldsymbol{\varepsilon} : \mathbb{C}_0 : \boldsymbol{\varepsilon} = \frac{1}{2} \boldsymbol{\varepsilon}_V^T \mathbf{D}_{ps} \boldsymbol{\varepsilon}_V$ .

## 2.3 Phase-Field Representation of Fracture

The sharp crack set is replaced by a continuous phase field:

$$\phi : \Omega \rightarrow [0, 1] \quad (9)$$

where  $\phi = 0$  represents the intact material and  $\phi = 1$  represents the fully damaged state. The diffuse crack width is controlled by the phase-field length scale  $l_0$ . Consequently, the crack is represented by a finite-width damaged zone rather than a geometrically sharp discontinuity.

## 2.4 Degradation of the Elastic Energy

The elastic energy is decomposed into an active component and a passive component:

$$\psi_e = \psi_e^+ + \psi_e^- \quad (10)$$

where  $\psi_e^+$  denotes the portion of the elastic energy permitted to drive fracture and  $\psi_e^-$  denotes the compressive portion that remains unaffected by degradation. The degraded elastic energy density is therefore written in generic form as:

$$\psi_e(\boldsymbol{\varepsilon}, \phi) = g(\phi)\psi_e^+(\boldsymbol{\varepsilon}) + \psi_e^-(\boldsymbol{\varepsilon}) \quad (11)$$

The corresponding degraded Cauchy stress is obtained from energetic differentiation:

$$\boldsymbol{\sigma} = \frac{\partial \psi_e}{\partial \boldsymbol{\varepsilon}} = g(\phi)\boldsymbol{\sigma}^+ + \boldsymbol{\sigma}^- \quad (12)$$

where  $\boldsymbol{\sigma}^\pm = \partial \psi_e^\pm / \partial \boldsymbol{\varepsilon}$ . The corresponding fourth-order consistent tangent moduli are:

$$\mathbb{C}_{\text{tan}} = \frac{\partial \boldsymbol{\sigma}}{\partial \boldsymbol{\varepsilon}} = g(\phi)\mathbb{C}^+ + \mathbb{C}^- \quad (13)$$

To prevent the singularity of the mechanical system in nearly fully damaged regions, the implementation introduces a residual stiffness parameter  $\kappa$  (typically set to  $10^{-8}$ ) through  $g_{\text{eff}}(\phi) = g(\phi) + \kappa$ .

## 2.5 Tension-Compression Energy Splits

PFCOMPAS-v1.0.0 supports three constitutive choices for the active/passive decomposition: isotropic degradation, the Amor volumetric-deviatoric split, and the Miehe spectral split.

### 2.5.1 Isotropic Degradation

For the isotropic formulation, the degradation acts upon the entirety of the strain energy density ( $\psi_e^+ = \psi_e$ ,  $\psi_e^- = 0$ ), yielding:

$$\psi_e(\boldsymbol{\varepsilon}, \phi) = g(\phi)\psi_e(\boldsymbol{\varepsilon}) \quad (14)$$

While computationally simple, this formulation permits unphysical degradation under pure compression.

### 2.5.2 Amor Volumetric-Deviatoric Split

For the Amor decomposition, the strain tensor is separated into volumetric and deviatoric parts ( $\boldsymbol{\varepsilon} = \boldsymbol{\varepsilon}_{\text{vol}} + \boldsymbol{\varepsilon}_{\text{dev}}$ ), with  $\boldsymbol{\varepsilon}_{\text{vol}} = \frac{1}{3}\text{tr}(\boldsymbol{\varepsilon})\mathbf{I}$ . The active and passive energy components are:

$$\psi_e^+ = \frac{1}{2}K\langle\text{tr}(\boldsymbol{\varepsilon})\rangle_+^2 + \mu(\boldsymbol{\varepsilon}_{\text{dev}} : \boldsymbol{\varepsilon}_{\text{dev}}) \quad (15)$$

$$\psi_e^- = \frac{1}{2}K\langle\text{tr}(\boldsymbol{\varepsilon})\rangle_-^2 \quad (16)$$

where  $K$  and  $\mu$  are the bulk and shear moduli. Volumetric compression is treated as purely passive.

### 2.5.3 Miehe Spectral Split

For the spectral formulation, the strain tensor is mapped to its principal directions via eigenvalue decomposition,  $\boldsymbol{\varepsilon} = \sum_{a=1}^d \lambda_a (\mathbf{n}_a \otimes \mathbf{n}_a)$ . The tensile and compressive parts are defined through the Macaulay brackets:

$$\boldsymbol{\varepsilon}^\pm = \sum_{a=1}^d \langle \lambda_a \rangle_\pm (\mathbf{n}_a \otimes \mathbf{n}_a) \quad (17)$$

The corresponding energy decomposition is:

$$\psi_e^\pm = \frac{\lambda}{2} \langle \text{tr}(\boldsymbol{\varepsilon}) \rangle_\pm^2 + \mu \text{tr} [(\boldsymbol{\varepsilon}^\pm)^2] \quad (18)$$

For algorithmic stability within the Newton solver, the Macaulay bracket is regularized by a dynamically scaled parameter  $\zeta = \zeta_0 \max(1, \|\boldsymbol{\varepsilon}\|_\infty)$ :

$$\langle x \rangle_{\pm, \zeta} = \frac{1}{2} \left( x \pm \sqrt{x^2 + \zeta^2} \right) \quad (19)$$

## 2.6 Generic Regularized Fracture Surface Energy

The generic regularized crack-density functional is written as:

$$\Gamma_{l_0}(\phi) = \frac{1}{c_0} \int_{\Omega} \left[ \frac{w(\phi)}{l_0} + l_0 |\nabla \phi|^2 \right] d\Omega \quad (20)$$

where  $w(\phi)$  is the crack-density function. For the AT2 formulation natively implemented in PFCOMPAS-v1.0.0,  $w(\phi) = \phi^2$  and  $c_0 = 2$ . Therefore, the fracture contribution to the total energy evaluates to:

$$E_{\text{fracture}} = \int_{\Omega} \left[ \frac{G_c}{2l_0} \phi^2 + \frac{G_c l_0}{2} |\nabla \phi|^2 \right] d\Omega \quad (21)$$

## 2.7 Total Potential Energy

The regularized total potential energy functional is defined as:

$$\Pi(\mathbf{u}, \phi) = E_{\text{elastic}}(\mathbf{u}, \phi) + E_{\text{fracture}}(\phi) - W_{\text{ext}}(\mathbf{u}) \quad (22)$$

The equilibrium state is sought as the constrained minimizer  $(\mathbf{u}, \phi) = \arg \min_{\mathbf{u}, \phi} \Pi(\mathbf{u}, \phi)$ , subject to the prescribed boundary conditions and the thermodynamic irreversibility constraint on  $\phi$ .

## 2.8 Governing Mechanical Equilibrium Equation

Variation of the total potential energy with respect to the displacement field gives the mechanical equilibrium equation:

$$\nabla \cdot \boldsymbol{\sigma} + \mathbf{b} = \mathbf{0} \quad \text{in } \Omega \quad (23)$$

The generic mechanical boundary conditions are  $\mathbf{u} = \bar{\mathbf{u}}$  on  $\Gamma_D^u$  and  $\boldsymbol{\sigma} \mathbf{n} = \bar{\mathbf{t}}$  on  $\Gamma_N^u$ . Under strict displacement-controlled loading parameterized by  $\lambda \in [0, 1]$ , the prescribed displacement follows a linear path  $\bar{\mathbf{u}}(\lambda) = \lambda \mathbf{u}_{\text{max}}$ .

## 2.9 Generic Phase-Field Evolution Equation

Variation of the total energy with respect to  $\phi$  gives the phase-field equilibrium equation:

$$g'(\phi)\mathcal{D} + \frac{G_c}{c_0 l_0} w'(\phi) - \frac{2G_c l_0}{c_0} \Delta\phi = 0 \quad \text{in } \Omega \quad (24)$$

where  $\mathcal{D}$  denotes the fracture-driving energetic quantity. To enforce thermodynamic irreversibility, PFCOMPAS-v1.0.0 drives the equation utilizing a strictly monotonic history variable  $\mathcal{H}$  rather than the instantaneous active elastic energy  $\psi_e^+$ . Specializing for the AT2 model ( $w(\phi) = \phi^2$ ,  $c_0 = 2$ ) yields:

$$g'(\phi)\mathcal{H} + \frac{G_c}{l_0} \phi - G_c l_0 \Delta\phi = 0 \quad (25)$$

## 2.10 Thermodynamic Irreversibility

Fracture evolution is irreversible and therefore satisfies

$$\dot{\phi}(\mathbf{x}, \lambda) \geq 0. \quad (26)$$

Equivalently, for two successive accepted states,

$$\phi_{n+1}(\mathbf{x}) \geq \phi_n(\mathbf{x}). \quad (27)$$

PFCOMPAS-v1.0.0 enforces this irreversibility through the history field associated with the tensile elastic energy density. The history field is defined as

$$\mathcal{H}(\mathbf{x}, \lambda) = \max_{\tilde{\lambda} \in [0, \lambda]} \psi_e^+(\mathbf{x}, \tilde{\lambda}), \quad (28)$$

where  $\psi_e^+$  denotes the tensile part of the elastic energy density. At each accepted state, the history field is updated according to

$$\mathcal{H}_{n+1}(\mathbf{x}) = \max(\mathcal{H}_n(\mathbf{x}), \psi_e^+(\mathbf{x}, \lambda_{n+1})), \quad (29)$$

which guarantees

$$\mathcal{H}_{n+1}(\mathbf{x}) \geq \mathcal{H}_n(\mathbf{x}). \quad (30)$$

At the discrete finite element level, the irreversibility of the phase field is additionally enforced by imposing an absolute lower bound based on the previously accepted state,

$$\phi_{n+1}(\mathbf{x}) \geq \phi_{\min}(\mathbf{x}) = \phi_n(\mathbf{x}). \quad (31)$$

Thus, both the history-field update and the discrete lower-bound constraint ensure that the phase field can only evolve in the direction of increasing fracture degradation.

## 2.11 Boundary and Initial Conditions

The global boundaries are decoupled into Dirichlet and Neumann sets for both fields:  $\partial\Omega = \Gamma_D^u \cup \Gamma_N^u$  and  $\partial\Omega = \Gamma_D^\phi \cup \Gamma_N^\phi$ . The natural phase-field boundary condition is  $\nabla\phi \cdot \mathbf{n} = 0$  on  $\Gamma_N^\phi$ . For domains containing pre-existing discrete cracks, the topology is initialized by solving the unloaded regularized diffusion equation ( $\phi - l_0^2 \Delta\phi = 0$ ) subject to the strict constraint  $\phi = 1$  on  $\Gamma_{\text{crack}}$ .

## 2.12 Specific PFCOMPAS-v1.0.0 Model Choices

To summarize, the general mathematical formulation is explicitly specialized in the current PFCOMPAS-v1.0.0 implementation via the following strict architecture constraints:

- **Kinematics:**  $\boldsymbol{\varepsilon} = \frac{1}{2}(\nabla \mathbf{u} + \nabla \mathbf{u}^T)$ , utilizing Engineering Shear Voigt notation.
- **Two-Dimensional Assumption:** Strict plane strain.
- **Fracture Model:** AT2 ( $w(\phi) = \phi^2$ ,  $c_0 = 2$ ).
- **Residual Stiffness:**  $g_{\text{eff}}(\phi) = g(\phi) + 10^{-8}$ .
- **Thermodynamic Driving Force:** Driven entirely by the history field  $\mathcal{H}$ .

## 2.13 Variational Structure of the Numerical Solution

The coupled potential energy functional  $\Pi(\mathbf{u}, \phi)$  is non-convex with respect to both variables simultaneously. Consequently, PFCOMPAS-v1.0.0 enforces a staggered alternate minimization architecture. For fixed  $\phi$ , the mechanical sub-problem minimizes  $\Pi$  with respect to  $\mathbf{u}$ . Subsequently, for fixed  $\mathbf{u}$  and history field  $\mathcal{H}$ , the active-set Newton solver minimizes  $\Pi$  with respect to  $\phi$ , subject to the irreversibility constraint  $\phi \geq \phi_{\min}$ .

# 3 PFCOMPAS-v1.0.0: Solver Architecture and Computational Algorithm

## 3.1 Overview

PFCOMPAS-v1.0.0 is a mesh-based finite-element framework developed for the numerical solution of quasi-static brittle fracture using the variational phase-field formulation. The present implementation employs the AT2 regularization and solves the coupled displacement-phase-field problem using an incremental staggered solution strategy.

The primary unknowns are the displacement field  $\mathbf{u}(\mathbf{x})$  and the scalar phase-field variable  $\phi(\mathbf{x}) \in [0, 1]$ , where  $\phi = 0$  represents an intact material state and  $\phi = 1$  represents a fully damaged or fractured state.

The computational framework combines four principal numerical components:

1. Nonlinear mechanical equilibrium
2. Phase-field fracture evolution
3. Enforcement of fracture irreversibility
4. Adaptive load stepping

The overall algorithm is implemented through a strict distinction between the previously *accepted state* and the current *trial state*. This distinction is particularly important for adaptive load stepping, since a failed trial must be discarded without modifying the previously converged solution.

## 3.2 Incremental Loading and Accepted States

The applied displacement is parameterized by a scalar load parameter  $\lambda \in [0, 1]$ . The prescribed displacement is therefore written as:

$$\bar{\mathbf{u}}(\lambda) = \lambda \mathbf{u}_{\max} \quad (32)$$

At load step  $n$ , the converged state is denoted by  $(\mathbf{u}_n, \phi_n, \mathcal{H}_n, \lambda_n)$ . A trial load increment, floored by a minimum increment size  $\Delta\lambda_{\min}$ , is selected as:

$$\Delta\lambda_n = \min(\Delta\lambda, \lambda_{\text{end}} - \lambda_n) \quad (33)$$

giving  $\lambda_{\text{trial}} = \lambda_n + \Delta\lambda_n$ . The trial solution is strictly initialized from the accepted state:

$$\mathbf{u}^{(0)} = \mathbf{u}_n, \quad \phi^{(0)} = \phi_n, \quad \mathcal{H}^{(0)} = \mathcal{H}_n \quad (34)$$

The accepted state is not modified during the trial solution, providing the rollback mechanism required by the adaptive algorithm.

## 3.3 Staggered Solution Strategy

At each prescribed load level, the coupled displacement-phase-field problem is solved using alternate minimization. The two primary fields are not solved simultaneously; instead, one field is held fixed while the other is solved.

Let  $k$  denote the staggered iteration. The computational sequence is:

$$\phi^k \xrightarrow{\text{mechanical solve}} \mathbf{u}^{k+1} \xrightarrow{\text{history update}} \mathcal{H}^{k+1} \xrightarrow{\text{phase-field solve}} \phi^{k+1} \quad (35)$$

The individual subproblems are therefore formulated as:

$$\mathbf{R}_u(\mathbf{u}; \phi) = \mathbf{0}, \quad R_\phi(\phi; \mathbf{u}, \mathcal{H}) = 0 \quad (36)$$

## 3.4 Nonlinear Mechanical Equilibrium

The discrete mechanical equilibrium equation is written as  $\mathbf{R}_u(\mathbf{u}, \phi) = \mathbf{f}_{\text{int}}(\mathbf{u}, \phi) - \mathbf{f}_{\text{ext}} = \mathbf{0}$ . For a fixed phase field, the Newton correction at iteration  $j$  is obtained from:

$$\mathbf{K}_u(\mathbf{u}^j, \phi) \Delta \mathbf{u}^j = -\mathbf{R}_u(\mathbf{u}^j, \phi) \quad (37)$$

followed by  $\mathbf{u}^{j+1} = \mathbf{u}^j + \Delta \mathbf{u}^j$ . The implementation solves only for the free displacement degrees of freedom ( $\mathbf{K}_{ff} \Delta \mathbf{u}_f = -\mathbf{R}_f$ ). The mechanical solver assembles the current tangent stiffness and internal force together at each Newton iteration.

### 3.4.1 Mechanical Newton Convergence

Two independent measures determine convergence. The normalized residual is:

$$r_u^j = \frac{\|\mathbf{R}_f^j\|}{R_{u,0}}, \quad \text{where } R_{u,0} = \max(\|\mathbf{R}_f^0\|, 1) \quad (38)$$

The normalized displacement correction is:

$$d_u^j = \frac{\|\Delta \mathbf{u}_f^j\|}{\max(\|\mathbf{u}_f^{j+1}\|, 1)} \quad (39)$$

Convergence strictly requires both  $r_u^j < \epsilon_{R,u} \wedge d_u^j < \epsilon_{\Delta u}$ . This dual criterion prevents a small displacement correction from being interpreted as equilibrium when a significant residual force remains.

### 3.5 History-Field Update

After the mechanical problem has converged, the tensile component of the elastic strain energy density is evaluated at the integration points. The history field is updated according to:

$$\mathcal{H}^{k+1}(\mathbf{x}) = \max [\mathcal{H}^k(\mathbf{x}), \psi_e^+(\mathbf{u}^{k+1})] \quad (40)$$

guaranteeing  $\mathcal{H}^{k+1}(\mathbf{x}) \geq \mathcal{H}^k(\mathbf{x})$  at every integration point.

### 3.6 Phase-Field Newton Problem & Active-Set Treatment

With  $\mathbf{u}$  and  $\mathcal{H}$  fixed, the phase-field equation is written as  $R_\phi(\phi; \mathbf{u}, \mathcal{H}) = 0$ . At Newton iteration  $j$ :

$$\mathbf{K}_\phi(\phi^j) \Delta \phi^j = -\mathbf{R}_\phi(\phi^j) \quad (41)$$

The lower-bound constraint ( $\phi \geq \phi_{\min}$ ) is enforced using a reduced-space active-set strategy. For a free phase-field degree of freedom  $i$ , the active set is identified as:

$$i \in \mathcal{A} \iff (\phi_i \leq \phi_{\min,i} + \epsilon_\phi) \wedge (R_{\phi,i} > 0) \quad (42)$$

If a degree of freedom is already at its lower bound and the unconstrained system would produce a negative increment, it is attempting to heal and is classified as active. The residual is projected such that active degrees of freedom receive zero Newton correction, and the system is solved strictly over the inactive set  $\mathcal{I}$ :

$$\mathbf{K}_{\phi,\mathcal{II}} \Delta \phi_{\mathcal{I}} = -\mathbf{R}_{\phi,\mathcal{I}} \quad (43)$$

#### 3.6.1 Phase-Field Newton Convergence

The normalized projected residual and the actual phase-field increment are defined as:

$$r_\phi^j = \frac{\|\mathbf{R}_\phi^{\text{proj},j}\|}{R_{\phi,0}}, \quad d_\phi^j = \frac{\|\Delta \phi_{\text{actual}}^j\|}{\max(\|\phi^{j+1}\|, 1)} \quad (44)$$

Both conditions must be satisfied ( $r_\phi^j < \epsilon_{R,\phi} \wedge d_\phi^j < \epsilon_{\Delta\phi}$ ).

### 3.7 Staggered Convergence

Following the displacement and phase-field solves, the change between successive staggered iterations is evaluated:

$$d_u^{\text{stag}} = \frac{\|\mathbf{u}^{n+1} - \mathbf{u}^n\|}{\max(\|\mathbf{u}^{n+1}\|, 1)}, \quad d_\phi^{\text{stag}} = \frac{\|\phi^{n+1} - \phi^n\|}{\max(\|\phi^{n+1}\|, 1)} \quad (45)$$

In the present implementation, the staggered convergence condition is based only on the displacement measure ( $d_u^{\text{stag}} < \epsilon_{\text{stag},u}$ ), as  $d_\phi^{\text{stag}}$  can be large during brutal crack growth.

### 3.8 Adaptive Load Stepping

The next load increment is determined from the numerical difficulty encountered during the previous step. The overall solver difficulty  $D_{\text{solver}}$  is:

$$D_{\text{solver}} = \max(D_{\text{stag}}, D_u, D_\phi) \quad (46)$$

where  $D_{\text{stag}}$ ,  $D_u$ , and  $D_\phi$  are normalized iteration counts. A separate measure accounts for the amount of phase-field evolution:

$$D_{\Delta\phi} = \frac{\max|\phi_{n+1} - \phi_n|}{\Delta\phi_{\text{target}}} \quad (47)$$

The total difficulty indicator is  $D = \max(D_{\text{solver}}, D_{\Delta\phi})$ . The load-step scaling factor is  $f = \min[1.5, \max(0.5, D^{-1/2})]$ , yielding the next increment:

$$\Delta\lambda_{n+1} = \text{clip}[\Delta\lambda_n f, \Delta\lambda_{\min}, \Delta\lambda_{\max}] \quad (48)$$

### 3.9 Load-Step Rejection and Rollback

A trial state must satisfy admissibility requirements. It is rejected if either Newton solve fails, or if the phase field changes excessively ( $\max|\phi_{\text{trial}} - \phi_{\text{accepted}}| > \Delta\phi_{\text{max}}^{\text{accepted}}$ ). In such cases, the increment is reduced and retried.

**Input:** Mesh, material, model, quadrature, BCs, and assembly data  
Initialize  $\mathbf{u}_0$ ,  $\phi_0$ ,  $\mathcal{H}_0$ , and  $\lambda_0$ ;  
Set initial load increment  $\Delta\lambda$ ;  
Store  $(\mathbf{u}_0, \phi_0, \mathcal{H}_0)$  as the accepted state;  
**while**  $\lambda < \lambda_{\text{end}}$  **do**  
    Construct  $\lambda_{\text{trial}} = \lambda + \min(\Delta\lambda, \lambda_{\text{end}} - \lambda)$ ;  
    Initialize trial state from accepted state ( $\phi_{\text{min}} = \phi_{\text{accepted}}$ );  
    **for**  $k = 1, \dots, N_{\text{stag}}^{\text{max}}$  **do**  
        Solve mechanical equilibrium using Newton iterations;  
        **if** *mechanical Newton fails* **then**  
            | Reject trial, reduce  $\Delta\lambda$ , and **continue** to next attempt;  
        **end**  
        Update history field:  $\mathcal{H} = \max(\mathcal{H}_{\text{old}}, \psi_e^+)$ ;  
        Solve phase-field problem using active-set Newton iterations;  
        **if** *phase-field Newton fails* **then**  
            | Reject trial, reduce  $\Delta\lambda$ , and **continue** to next attempt;  
        **end**  
        Evaluate staggered convergence;  
        **if** *staggered convergence satisfied* **then**  
            | **break**;  
        **end**  
    **end**  
    **if** *staggered convergence not satisfied* **or**  $\Delta\phi_{\text{max}} > \Delta\phi_{\text{max}}^{\text{accepted}}$  **then**  
        | Reject trial, reduce  $\Delta\lambda$ , and **continue**;  
    **end**  
    Compute energies, reaction forces, and solver difficulty  $D$ ;  
    Accept trial state:  $(\mathbf{u}, \phi, \mathcal{H}, \lambda) \leftarrow (\mathbf{u}_{\text{trial}}, \phi_{\text{trial}}, \mathcal{H}_{\text{trial}}, \lambda_{\text{trial}})$ ;  
    Update  $\Delta\lambda_{\text{new}} = \text{clip}(\Delta\lambda \cdot f, \Delta\lambda_{\text{min}}, \Delta\lambda_{\text{max}})$ ;  
    Perform stress recovery and error estimation, store accepted solution;  
**end**

**Algorithm 1:** Adaptive Staggered Solution Procedure

## 4 Advanced Constitutive Algorithms

### 4.1 Constitutive Degradation and Cohesive Zone Modeling

The mathematical bridge coupling the scalar damage field to the structural deformation of the solid is the constitutive degradation function  $g(\phi)$ . This monotonic function maps the intact state ( $\phi = 0$ ) to a fully pristine stiffness ( $g = 1$ ) and the completely fractured state ( $\phi = 1$ ) to a vanished load-bearing capacity ( $g \rightarrow 0$ ).

#### 4.1.1 The Role of Residual Stiffness

A critical numerical necessity in phase-field continuum damage mechanics is the inclusion of a residual stiffness parameter,  $\kappa$ . If an element becomes fully damaged such that  $g(\phi) = 0$ , the corresponding local stiffness matrix  $\mathbf{K}_e$  drops to exactly zero. Upon global assembly, this renders the structural stiffness matrix  $\mathbf{K}_u$  singular, causing the linear solver to crash during the mechanical Newton step.

The PFCOMPAS-v1.0.0 framework rigorously evades this catastrophic ill-conditioning by appending a vanishingly small artificial parameter (typically  $\kappa = 10^{-8}$ ) to all active degradation functions:

$$g_{\text{eff}}(\phi) = g(\phi) + \kappa \quad (49)$$

This unconditionally preserves the positive-definiteness of the global equilibrium system, even across completely fractured topologies.

#### 4.1.2 Beyond Quadratic: Delaying Crack Nucleation

The classical AT2 phase-field formulation utilizes a standard quadratic degradation function:  $g(\phi) = (1 - \phi)^2$ . While computationally elegant, this function possesses a profound physical limitation: because  $g'(0) \neq 0$ , the material begins to soften immediately upon the application of any infinitesimal load. This completely eliminates the strictly linear-elastic regime observed in brittle solids, causing unphysical, premature crack nucleation prior to reaching the material's critical fracture strength.

To circumvent this and establish a framework capable of Phase-Field Cohesive Zone Modeling (PF-CZM), PFCOMPAS-v1.0.0 provides a comprehensive library of higher-order and asymptotic degradation functions. These functions artificially enforce a nucleation threshold. For example, the cubic ( $3(1 - \phi)^2 - 2(1 - \phi)^3$ ) and quintic polynomials artificially flatten the slope of  $g(\phi)$  near  $\phi = 0$ . Because  $g'(0) = 0$  in these formulations, damage is suppressed until the local strain energy surpasses a critical energetic barrier, effectively mimicking the traction-separation laws of cohesive zone elements without altering the underlying mesh topology.

#### 4.1.3 Algorithmic Necessity of Exact Analytical Derivatives

The `degradationFunction` subroutine returns not only  $g(\phi)$ , but its exact first and second analytical derivatives with respect to the phase field,  $g'(\phi)$  and  $g''(\phi)$ .

- **The Driving Force ( $g'$ ):** The first derivative  $g'(\phi)$  serves as the energetic driving force within the local internal residual vector ( $\mathbf{R}_\phi$ ) of the phase-field evolution equation.
- **The Consistent Tangent ( $g''$ ):** The second derivative  $g''(\phi)$  is strictly required to formulate the exact Jacobian (the tangent stiffness matrix  $\mathbf{K}_\phi$ ) for the phase-field Newton-Raphson solver. Failure to include the exact analytical  $g''(\phi)$  results in a mismatched tangent, which instantly destroys the quadratic asymptotic convergence properties.

## 4.2 Tension-Compression Energy Splits and Asymmetry

A profound physical inadequacy of the pristine, unmodified AT2 formulation is its assumption of isotropic material degradation. If the degradation function is applied uniformly to the entire elastic strain energy density, the model erroneously permits crack nucleation and propagation under states of pure hydrostatic compression.

To rigorously enforce tension-compression asymmetry, PFCOMPAS-v1.0.0 natively implements strain energy decomposition schemes. The total elastic energy is partitioned into an active tensile component ( $\psi_e^+$ ) and a passive compressive component ( $\psi_e^-$ ):

$$\psi_e(\boldsymbol{\varepsilon}, \phi) = g(\phi)\psi_e^+(\boldsymbol{\varepsilon}) + \psi_e^-(\boldsymbol{\varepsilon}) \quad (50)$$

By deriving the stress tensor  $\boldsymbol{\sigma} = \partial\psi_e/\partial\boldsymbol{\varepsilon}$ , the degradation is strictly confined to the tensile stress fields, completely preserving the compressive stiffness of the solid.

#### 4.2.1 The Amor (Volumetric-Deviatoric) Split

The computationally elegant split proposed by Amor et al. assumes that crack propagation is driven by volumetric expansion and deviatoric shear. The strain tensor is decomposed into  $\boldsymbol{\varepsilon} = \boldsymbol{\varepsilon}_{\text{vol}} + \boldsymbol{\varepsilon}_{\text{dev}}$ . The energy components are derived utilizing the bulk modulus  $K$  and shear modulus  $\mu$ :

$$\psi_e^+ = \frac{1}{2}K\langle\text{tr}(\boldsymbol{\varepsilon})\rangle_+^2 + \mu\text{tr}(\boldsymbol{\varepsilon}_{\text{dev}}^2) \quad (51)$$

$$\psi_e^- = \frac{1}{2}K\langle\text{tr}(\boldsymbol{\varepsilon})\rangle_-^2 \quad (52)$$

Because this formulation avoids eigenvalue decompositions, it is computationally rapid.

#### 4.2.2 The Miehe Spectral Split

To achieve maximum physical fidelity, the framework implements the rigorous spectral decomposition formulated by Miehe et al. The strain tensor is mapped to its principal directions via eigenvalue decomposition:

$$\boldsymbol{\varepsilon} = \sum_{I=1}^{\text{dim}} \varepsilon_I (\mathbf{n}_I \otimes \mathbf{n}_I) \quad (53)$$

The positive and negative principal strains are isolated using the Macaulay bracket  $(\langle\varepsilon_I\rangle_{\pm})$ . The split tensors are then reconstructed:

$$\boldsymbol{\varepsilon}_{\pm} = \sum_{I=1}^{\text{dim}} \langle\varepsilon_I\rangle_{\pm} (\mathbf{n}_I \otimes \mathbf{n}_I) \quad (54)$$

This formulation strictly prohibits crack face interpenetration, making it mandatory for simulating complex shear-dominated or mixed-mode fracture phenomena.

#### 4.2.3 Algorithmic Regularization of the Macaulay Bracket

A profound computational challenge associated with both the Amor and Spectral split schemes is the inherent non-differentiability of the standard Macaulay bracket,  $\langle x \rangle_{\pm} = \frac{1}{2}(x \pm |x|)$ . The absolute value operator  $|x|$  possesses a sharp derivative discontinuity at exactly  $x = 0$ . Consequently, the 4th-order tangent stiffness moduli  $\mathbb{C}_{\text{tan}} = \partial^2\psi_e/\partial\boldsymbol{\varepsilon}^2$  become mathematically undefined at the zero-strain transition boundary, which can destroy the solver's convergence.

#### 4.2.4 Hyperbolic Smoothing for $C^1$ -Continuity

To recover an unconditionally stable tangent stiffness matrix, the PFCOMPAS-v1.0.0 framework implements an algorithmic, hyperbolic smoothing of the Macaulay bracket:

$$\langle x \rangle_{\pm, \zeta} = \frac{1}{2} \left( x \pm \sqrt{x^2 + \zeta^2} \right) \quad (55)$$

This continuously differentiable approximation yields a well-defined first derivative:

$$\frac{\partial \langle x \rangle_{\pm, \zeta}}{\partial x} = \frac{1}{2} \left( 1 \pm \frac{x}{\sqrt{x^2 + \zeta^2}} \right) \quad (56)$$

As  $\zeta \rightarrow 0$ , the smoothed derivative seamlessly recovers the Heaviside step function behavior without ever triggering a division-by-zero singularity.

#### 4.2.5 Eigenvalue Derivatives for the Spectral Tangent

In the spectral representation, the active (tensile) principal stress eigenvalues  $\sigma_I^+$  are fundamentally defined by the smoothed active principal strains:

$$\sigma_I^+ = \lambda \langle \text{tr}(\boldsymbol{\varepsilon}) \rangle_{+, \zeta} + 2\mu \langle \varepsilon_I \rangle_{+, \zeta} \quad (57)$$

To assemble the 4th-order consistent tangent moduli, the solver must evaluate  $\partial \sigma_I^+ / \partial \varepsilon_I$ . The `spectralDerivative` subroutine explicitly computes this scalar derivative:

$$f'_+(\varepsilon_I) = \frac{\partial \langle \varepsilon_I \rangle_{+, \zeta}}{\partial \varepsilon_I} = \frac{1}{2} \left( 1 + \frac{\varepsilon_I}{\sqrt{\varepsilon_I^2 + \zeta^2}} \right) \quad (58)$$

$$f'_-(\varepsilon_I) = \frac{\partial \langle \varepsilon_I \rangle_{-, \zeta}}{\partial \varepsilon_I} = \frac{1}{2} \left( 1 - \frac{\varepsilon_I}{\sqrt{\varepsilon_I^2 + \zeta^2}} \right) \quad (59)$$

#### 4.2.6 Dynamic Scale Invariance

To guarantee floating-point robustness across all deformation regimes, the framework scales the regularization parameter  $\zeta$  dynamically against the infinite norm of the local strain tensor:

$$\zeta = \zeta_0 \times \max(1.0, \|\boldsymbol{\varepsilon}\|_\infty) \quad (60)$$

By clamping the lower bound multiplier to 1.0, the solver prevents  $\zeta$  from dropping to identically zero in pristine elements.

#### 4.2.7 Algorithmic Extraction of the Amor Consistent Tangent

For the Amor split, the exact active 4th-order moduli are derived from the smoothed active stress tensor:

$$\mathbb{C}^+ = \frac{\partial \boldsymbol{\sigma}^+}{\partial \boldsymbol{\varepsilon}} = K \left( \frac{\partial \langle \text{tr}(\boldsymbol{\varepsilon}) \rangle_{+, \zeta}}{\partial \text{tr}(\boldsymbol{\varepsilon})} \right) \mathbf{I} \otimes \mathbf{I} + 2\mu \left( \mathbb{I}^{\text{sym}} - \frac{1}{3} \mathbf{I} \otimes \mathbf{I} \right) \quad (61)$$

where  $\mathbb{I}^{\text{sym}}$  is the 4th-order symmetric identity tensor.

#### 4.2.8 Volumetric Tangent Extraction for Passive Compression

The exact passive 4th-order consistent tangent  $\mathbb{C}^-$  is purely dilatational:

$$\mathbb{C}^- = \frac{\partial \boldsymbol{\sigma}^-}{\partial \boldsymbol{\varepsilon}} = K \left( \frac{\partial \langle \text{tr}(\boldsymbol{\varepsilon}) \rangle_{-, \zeta}}{\partial \text{tr}(\boldsymbol{\varepsilon})} \right) \mathbf{I} \otimes \mathbf{I} \quad (62)$$

The framework executes the `buildVolumetricTangent` subroutine to probe the analytical stress equation with discrete Voigt basis tensors, mapping the dilatational tensor directly into the finite element Voigt architecture.

#### 4.2.9 Kinematic Perturbation via Voigt Basis Tensors

Rather than hard-coding explicitly expanded 4th-order tensor components, PFCOMPAS-v1.0.0 employs a programmatic kinematic perturbation technique within the `buildAmorTangent` subroutine. The solver sequentially generates a 2nd-order symmetric unit basis tensor  $\mathbf{H}^{(j)}$  corresponding to the  $j$ -th Voigt engineering strain direction and evaluates a highly specific stress perturbation tensor  $\Delta\boldsymbol{\sigma}^{(j)}$ :

$$\Delta\boldsymbol{\sigma}^{(j)} = K \left[ \frac{\partial \langle \text{tr}(\boldsymbol{\varepsilon}) \rangle_{+, \zeta}}{\partial \text{tr}(\boldsymbol{\varepsilon})} \text{tr}(\mathbf{H}^{(j)}) \right] \mathbf{I} + 2\mu \mathbf{H}_{\text{dev}}^{(j)} \quad (63)$$

This tensor is systematically mapped to the  $j$ -th column of the 2D tangent moduli matrix  $\mathbf{C}_{\text{Voigt}}$ .

#### 4.2.10 Kinematic Basis Tensors and Engineering Shear Mapping

The `voigtBasisTensor` subroutine rigorously enforces the engineering shear strain convention ( $\gamma = 2\varepsilon$ ). A unit perturbation in the Voigt engineering shear strain translates to a physical tensorial shear perturbation of exactly half that magnitude. Consequently, the basis tensor is explicitly defined with 0.5 off-diagonal elements:

$$\mathbf{H}_{2\text{D}}^{(3)} = \begin{bmatrix} 0 & 0.5 & 0 \\ 0.5 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \quad (64)$$

#### 4.2.11 Fréchet Derivatives and the Spectral Consistent Tangent

Developing the exact 4th-order consistent tangent stiffness tensor for the spectral decomposition is mathematically demanding because a perturbation in the global strain tensor  $\Delta\boldsymbol{\varepsilon}$  induces highly coupled rotations of the principal strain directions (eigenvectors  $\mathbf{n}_a$ ).

#### 4.2.12 Derivatives of Isotropic Tensor Functions

The active and passive strain tensors are isotropic tensor functions. Evaluating the Fréchet derivative in the direction of the perturbation  $\mathbf{H}$  yields:

$$\Delta\boldsymbol{\varepsilon}_{\pm} = \sum_{a=1}^3 \sum_{b=1}^3 C_{ab}^{\pm} [(\mathbf{n}_a \otimes \mathbf{n}_a) \mathbf{H} (\mathbf{n}_b \otimes \mathbf{n}_b)] \quad (65)$$

where  $C_{ab}^{\pm}$  is the scalar coefficient matrix governing the coupled stretch/rotation derivatives.

#### 4.2.13 Divided Differences and Eigenvector Rotation

The `buildSpectralTangent` subroutine evaluates the scalar coefficients  $C_{ab}^{\pm}$  using divided differences. For co-axial (normal) components ( $a = b$ ):

$$C_{aa}^{\pm} = \frac{\partial \langle \lambda_a \rangle_{\pm, \zeta}}{\partial \lambda_a} = f'_{\pm}(\lambda_a) \quad (66)$$

For cross-coupled (shear) components ( $a \neq b$ ):

$$C_{ab}^{\pm} = \frac{\langle \lambda_a \rangle_{\pm, \zeta} - \langle \lambda_b \rangle_{\pm, \zeta}}{\lambda_a - \lambda_b} \quad (67)$$

#### 4.2.14 L'Hôpital's Rule for Repeated Eigenvalues

Under states of pure isotropic tension or compression (hydrostatic stress), the principal strains are identical ( $\lambda_a = \lambda_b$ ), rendering the denominator of the divided difference identically zero. The framework dynamically monitors the spectral gap ( $\Delta\lambda = \lambda_a - \lambda_b$ ). If it falls below a machine-precision tolerance, the solver invokes L'Hôpital's rule:

$$\lim_{\lambda_a \rightarrow \lambda_b} \frac{f(\lambda_a) - f(\lambda_b)}{\lambda_a - \lambda_b} = f' \left( \frac{\lambda_a + \lambda_b}{2} \right) \quad (68)$$

guaranteeing quadratic convergence across all arbitrary multiaxial stress states.

## 5 Spatial Discretization and Finite Elements

### 5.1 Motivation for a Unified Mesh Database

In standard, single-field finite element codes, basic arrays containing nodal coordinates and element connectivity are often sufficient for the assembly of global stiffness matrices. However, the simulation of phase-field fracture necessitates a coupled, multi-field numerical framework. Because the solution scheme employs an alternate minimization (staggered) approach, where the balance of linear momentum and the damage evolution equation are solved sequentially, the underlying mesh database must be highly structured. The `buildMeshStruct2D` function serves as the foundational pre-processor of this MATLAB framework, transforming raw structural data into an optimized, unified structure that drives the rest of the solver.

#### 5.1.1 Explicit Degree of Freedom (DOF) Mapping

The most critical operation performed during the mesh initialization is the precomputation of the Degree of Freedom (DOF) mapping. The AT2 phase-field model features two distinct fields:

- The vector displacement field,  $\mathbf{u}$ , possessing two DOFs ( $u_x$  and  $u_y$ ) per node in a 2D domain.
- The scalar phase-field damage variable,  $\phi$ , possessing a single DOF per node.

To avoid the computational overhead of calculating global matrix indices dynamically during the highly non-linear iterative solution phase, `buildMeshStruct2D` pre-allocates these mappings into `mesh.elem.dofU` and `mesh.elem.dofP`. The displacement DOFs are structured such that odd indices correspond to the  $x$ -direction and even indices correspond to the  $y$ -direction. This explicit separation is mandatory for passing correctly dimensioned local stiffness matrices to the global assembly routines, especially when evaluating complex phenomena like spectral tension-compression splits which rely entirely on the displacement field gradients.

#### 5.1.2 High-Performance Sparse Matrix Pre-Allocation

A notorious computational bottleneck in MATLAB-based finite element frameworks is the dynamic allocation of global sparse matrices during the assembly phase. In a staggered phase-field solver, both the displacement stiffness matrix ( $\mathbf{K}_u$ ) and the damage stiffness

matrix ( $\mathbf{K}_\phi$ ) must be entirely reassembled at every single Newton-Raphson iteration due to the highly non-linear evolution of the degradation function  $g(\phi)$ .

To circumvent this, the PFCOMPAS-v1.0.0 framework rigorously decouples the topological matrix mapping from the geometric and constitutive integrations. The `buildAssemblyPattern` function executes exclusively during the pre-processing phase. It iterates through the pre-mapped DOFs to generate static global row (**I**) and column (**J**) index arrays for both the vector and scalar fields using Cartesian products (`ndgrid`). This guarantees that during the active non-linear staggered solve, the framework can leverage MATLAB's highly optimized, built-in sparse constructor.

### 5.1.3 Parallel Element Evaluation and Vectorized Assembly

To fully exploit modern multi-core processor architectures, PFCOMPAS-v1.0.0 rigorously decouples the numerical integration phase from the global assembly phase. During initialization, a dense array `V_K` of dimensions  $[n_{en}^2 \times n_{elem}]$  is pre-allocated.

A parallel `parfor` loop is then invoked, distributing the computationally intensive numerical integrations across the available CPU workers. Each worker calculates the local matrix  $\mathbf{K}_e$  and writes the flattened result strictly to its assigned, independent column slice (`V_K(:,e)`). Following the completion of the parallel loop, the framework executes a single, highly vectorized call to MATLAB's native sparse constructor:

```
A = sparse(I, J, V_K(:), nnodes, nnodes);
```

The `sparse()` algorithm accumulates values sharing identical global row and column identifiers, effectively handling the global node sharing automatically.

### 5.1.4 Algorithmic Requirements for Verfürth Estimators

The implementation of explicit *a posteriori* error estimators, such as those formulated by Verfürth, introduces a fundamentally different topological requirement. Verfürth estimators evaluate the quality of the discrete solution by computing jump residuals across element interfaces: the jump in the traction vector  $[[\boldsymbol{\sigma} \cdot \mathbf{n}]]$  for the momentum balance, and the jump in the damage gradient  $[[\nabla\phi \cdot \mathbf{n}]]$  for the phase-field equation. Consequently, the solver must possess an explicit mapping of the mesh's internal edges.

### 5.1.5 Hash-Map Based Interface Detection

To construct this edge database without incurring a prohibitive  $\mathcal{O}(N^2)$  computational penalty, the `buildEdgeConnectivity` function utilizes a hash-map (dictionary) approach via MATLAB's `containers.Map`. The algorithm extracts the global node identifiers for the local edges, sorts the node pairs in ascending order to guarantee direction-independence, and converts them into unique character string keys (e.g., '14\_25').

### 5.1.6 Data Structure for Jump Residuals

As the mesh is parsed, the size of the array stored at each key dictates the topological nature of the edge:

- **Boundary Edges (Array Size = 1):** The edge resides on the external boundary and is subjected to Neumann or Dirichlet boundary conditions.

- **Interior Edges (Array Size = 2):** The function maps the “left” and “right” sharing elements (`elemL`, `elemR`) along with their respective local edge numbers (`locL`, `locR`).

This resulting [`elemL`, `locL`, `elemR`, `locR`] matrix structure is the exact architectural prerequisite required by the residual evaluation subroutines.

### 5.1.7 Geometric Boundary Classification

Applying Dirichlet boundary conditions manually can be cumbersome. The mesh builder automates this process by employing a bounding-box algorithm. Using a strict Cartesian tolerance of  $10^{-8}$ , the code geometrically parses the exterior boundaries (identified where the element sharing count is zero) and classifies them into `top`, `bottom`, `left`, and `right` sets.

## 5.2 Element Interpolation and Lagrange Basis Functions

In the finite element method, the continuous displacement field  $\mathbf{u}(\mathbf{x})$  and the scalar phase-field  $\phi(\mathbf{x})$  are approximated over discrete elements using localized polynomial basis functions. The PFCOMPAS-v1.0.0 framework accommodates an extensive suite of these interpolation functions via the `lagrange_basis` subroutine.

### 5.2.1 The Role of Shape Functions and Reference Coordinates

To bypass the mathematical complexity of differentiating arbitrary, distorted geometries in physical space, the framework defines all shape functions strictly in a standardized reference (parent) coordinate system  $(\xi, \eta, \zeta)$ . The `lagrange_basis` function takes the reference coordinates of a specified Gauss integration point and returns both the shape function array  $\mathbf{N}(\xi, \eta, \zeta)$  and the local parametric derivative matrix  $\nabla_{\xi}\mathbf{N}$ .

### 5.2.2 The Necessity of Higher-Order Elements in Phase-Field Fracture

While standard linear elements are computationally simple, their  $C^0$ -continuous, linear interpolation spaces present a severe geometric challenge for the AT2 phase-field model. The exact analytical solution for a fully formed, stationary phase-field crack profile in 1D follows an exponential decay:

$$\phi(x) = \exp\left(-\frac{|x|}{l_0}\right) \quad (69)$$

Attempting to accurately resolve this infinitely differentiable exponential curve utilizing piece-wise linear finite elements induces severe discretization error. To circumvent this, the expanded `lagrange_basis` library incorporates higher-order elements, including quadratic (Q8, Q9, T6, H10) and cubic (Q16, H20) families. By deploying higher-order spatial interpolation, the solver accurately captures steep phase-field gradients with significantly fewer elements across the characteristic length scale  $l_0$ .

## 5.3 Kinematics and the Strain-Displacement Operator

In the context of infinitesimal elasticity, the kinematic relationship between the displacement field  $\mathbf{u}(\mathbf{x})$  and the symmetric small-strain tensor  $\boldsymbol{\varepsilon}$  is governed by the symmetric

gradient operator:

$$\boldsymbol{\varepsilon} = \frac{1}{2} [\nabla \mathbf{u} + (\nabla \mathbf{u})^T] \quad (70)$$

### 5.3.1 Voigt Notation and Engineering Shear Strains

The `displacementBMatrix` subroutine constructs the discrete strain-displacement operator,  $\mathbf{B}$ . By evaluating the physical spatial derivatives of the shape functions  $(\nabla_{\mathbf{x}} \mathbf{N})$ , the  $\mathbf{B}$ -matrix maps the discrete nodal displacement vector  $\mathbf{u}_e$  directly to the continuous Voigt strain vector:

$$\boldsymbol{\varepsilon}_{\text{Voigt}} = \mathbf{B} \mathbf{u}_e \quad (71)$$

A critical convention enforced within this subroutine is the explicit use of engineering shear strains  $(\gamma_{ij} = 2\varepsilon_{ij})$  rather than tensorial shear strains. For a 2D configuration, the explicit mathematical structure of the  $\mathbf{B}$ -matrix for a given node  $I$  is assembled as:

$$\mathbf{B}_I = \begin{bmatrix} \frac{\partial N_I}{\partial x} & 0 \\ 0 & \frac{\partial N_I}{\partial y} \\ \frac{\partial N_I}{\partial y} & \frac{\partial N_I}{\partial x} \end{bmatrix} \quad (72)$$

### 5.3.2 Work-Conjugate Stress Mapping

To close the mechanical loop during the evaluation of the internal residual vector  $\mathbf{R}_e = \int_{\Omega_e} \mathbf{B}^T \boldsymbol{\sigma}_{\text{Voigt}} d\Omega$ , the degraded Cauchy stress tensor  $\boldsymbol{\sigma}$  must be flattened into a 1D Voigt vector via the `stressTensorToVoigt` subroutine. A paramount requirement is the preservation of work-conjugacy:

$$\delta W_{\text{int}} = \boldsymbol{\sigma} : \delta \boldsymbol{\varepsilon} \equiv \boldsymbol{\sigma}_{\text{Voigt}}^T \delta \boldsymbol{\varepsilon}_{\text{Voigt}} \quad (73)$$

Expanding the tensorial double contraction dictates that to remain mathematically equivalent alongside engineering shear strains, the shear component of the stress Voigt vector must simply be the raw Cauchy shear stress  $(\tau_{xy} = \sigma_{xy})$ .

### 5.3.3 Pristine Isotropic Elasticity and Energy Equivalence

Prior to degradation, the fundamental mechanical response of the intact material is governed by the 4th-order isotropic elasticity tensor  $\mathbb{C}_0$ . Within the discrete architecture, this tensor is mapped into a 2D matrix  $\mathbf{D}$  via the `matStiff` subroutine.

### 5.3.4 Plane Strain Enforcement in 2D Phase-Field

For 2D simulations, the framework explicitly enforces a plane strain configuration:

$$\mathbf{D}_{2D} = \frac{E}{(1+\nu)(1-2\nu)} \begin{bmatrix} 1-\nu & \nu & 0 \\ \nu & 1-\nu & 0 \\ 0 & 0 & \frac{1-2\nu}{2} \end{bmatrix} \quad (74)$$

By enforcing strict plane strain, the 2D domain rigorously represents an infinitely thick slice of material, ensuring the fracture energy  $G_c$  is correctly dissipated exclusively via in-plane damage evolution without unconstrained out-of-plane Poisson contraction.

### 5.3.5 Thermodynamic Consistency of Shear Moduli

Because the framework utilizes engineering shear strains, the elastic strain energy density evaluated via Voigt vectors must remain thermodynamically identical to its tensorial equivalent:

$$W = \frac{1}{2} \boldsymbol{\varepsilon} : \mathbb{C}_0 : \boldsymbol{\varepsilon} \equiv \frac{1}{2} \boldsymbol{\varepsilon}_{\text{Voigt}}^T \mathbf{D} \boldsymbol{\varepsilon}_{\text{Voigt}} \quad (75)$$

To preserve this equality, the diagonal shear entries of the  $\mathbf{D}$  matrix must strictly equal the shear modulus  $\mu$ .

## 5.4 Numerical Integration and Quadrature Schemes

The accurate numerical evaluation of the weak form integrals is fundamentally governed by the underlying quadrature scheme, accommodated via the `quadrature.m` routine.

### 5.4.1 The Necessity of High-Order Quadrature in Phase-Field Modeling

The introduction of the degradation function,  $g(\phi)$ , severely non-linearizes the elemental stiffness matrix. If users test highly non-linear degradation functions (e.g., quartic or exponential models), the spatial variation of  $g(\phi)$  within the length scale  $l_0$  becomes immensely steep. Standard low-order quadrature will severely under-integrate the elastic strain energy density. By providing dynamically scalable Gauss quadrature rules up to order 16, the framework empowers researchers to rigorously verify integration sufficiency.

### 5.4.2 Multidimensional Tensor Products and Normalization

For quadrilateral and hexahedral elements, the multidimensional Gauss points and weights are generated natively via tensor products of 1D reference points mapped over the domain  $[-1, 1]$ . For triangular and tetrahedral integrations, standard tabulated weights (which sum to unity) are normalized strictly to match the geometric reference areas/volumes (e.g., divided by 2 for the reference triangle).

### 5.4.3 1D Quadrature for Boundary and Interface Integration

Phase-field formulations enriched with explicit error diagnostics strictly require dedicated 1D integration schemes via the `Gauss1D.m` subroutine. This fulfills two critical requirements within a 2D spatial discretization: integrating applied surface tractions along external edges ( $\partial\Omega_N$ ) and computing jump residuals ( $[[\boldsymbol{\sigma} \cdot \mathbf{n}]]$  and  $[[\nabla\phi \cdot \mathbf{n}]]$ ) across internal element interfaces for Verfürth estimators.

## 5.5 Phase-Field Initialization and Crack Regularization

A fundamental challenge in implementing phase-field fracture is the handling of pre-existing crack topologies. The AT2 variational model mandates that the crack be represented as a continuous, diffuse band of degraded material. If a simulation were to commence with a sharp nodal jump, the extreme spatial gradients would instantly generate massive, unphysical residual forces, triggering immediate solver divergence.

### 5.5.1 The Homogeneous Screened Poisson Equation

To physically transition a sharp discrete crack into a mathematically valid AT2 continuum field, the `initializePhaseField` subroutine executes a rigorous initialization. The static, fully unloaded diffuse crack profile is governed entirely by the fracture surface energy functional, yielding the homogeneous screened Poisson equation:

$$\phi - l_0^2 \Delta \phi = 0 \quad \text{in } \Omega \quad (76)$$

subject to the strict Dirichlet boundary condition representing the pre-existing sharp defect ( $\phi(\mathbf{x}) = 1$  for  $\mathbf{x} \in \Gamma_{\text{crack}}$ ).

### 5.5.2 Matrix Partitioning and Linear Solve

Through standard Galerkin finite element discretization, this translates into a global linear algebraic system  $\mathbf{A}\phi = \mathbf{0}$ , where the initialization matrix is  $\mathbf{A} = \mathbf{M} + l_0^2 \mathbf{K}_{\text{diff}}$ . The framework partitions the degrees of freedom into free ( $f$ ) and constrained ( $c$ ) sets:

$$\begin{bmatrix} \mathbf{A}_{ff} & \mathbf{A}_{fc} \\ \mathbf{A}_{cf} & \mathbf{A}_{cc} \end{bmatrix} \begin{Bmatrix} \phi_f \\ \phi_c \end{Bmatrix} = \begin{Bmatrix} \mathbf{0} \\ \mathbf{0} \end{Bmatrix} \quad (77)$$

yielding a symmetric, positive-definite reduced system,  $\mathbf{A}_{ff}\phi_f = -\mathbf{A}_{fc}\phi_c$ , which is rapidly solved via Cholesky or LU decomposition.

### 5.5.3 Numerical Bounding and Symmetry Enforcement

To filter out floating-point interpolation artifacts, the framework enforces a strict numerical tolerance clamp ( $10^{-12}$ ) ensuring the initial state is strictly bounded within  $[0, 1]$ . Furthermore, accumulating floating-point numbers across out-of-order parallel threads introduces infinitesimal numerical asymmetries. The framework applies a strict post-assembly symmetry enforcement:

$$\mathbf{A} \leftarrow \frac{1}{2}(\mathbf{A} + \mathbf{A}^T) \quad (78)$$

guaranteeing that highly optimized symmetric linear solvers are consistently triggered.

### 5.5.4 Element-Level Weak Form and Isoparametric Mapping

By multiplying by a scalar test function  $\delta\phi$  and applying the divergence theorem, the continuum weak form is obtained:

$$\int_{\Omega_e} \delta\phi \phi \, d\Omega + l_0^2 \int_{\Omega_e} \nabla(\delta\phi) \cdot \nabla\phi \, d\Omega = 0 \quad (79)$$

Discretizing using local shape functions ( $\phi \approx \mathbf{N}^T \phi_e$  and  $\nabla\phi \approx \mathbf{B}_\phi^T \phi_e$ ) reduces the weak form to a local linear matrix:

$$\mathbf{K}_e^\phi = \mathbf{M}_e + l_0^2 \mathbf{K}_{\text{diff},e} \quad (80)$$

### 5.5.5 Numerical Integration via Jacobian Transformation

At each Gauss quadrature point  $q$ , the transformation to physical spatial derivatives  $\mathbf{B}_\phi$  requires the Jacobian matrix  $\mathbf{J} = \mathbf{X}^T \nabla_\xi \mathbf{N}$ . The elemental matrix is ultimately computed via Gauss-Legendre numerical summation:

$$\mathbf{K}_e^\phi = \sum_{q=1}^{n_{gp}} (\mathbf{N}_q \mathbf{N}_q^T + l_0^2 \mathbf{B}_{\phi,q} \mathbf{B}_{\phi,q}^T) W_q |\mathbf{J}_q| \quad (81)$$

Because this initialization routine governs solely geometric regularized diffusion, it inherently bypasses all complex material constitutive tensors.

## 6 Main Execution Script and Global Architecture

The core architecture of the PFCOMPAS-v1.0.0 framework is driven by a master execution script (`main.m`). Having established the continuum mathematical formulation and the non-linear solver logic in the preceding chapters, this section details how the MATLAB codebase technically orchestrates these algorithms to achieve high-performance, vectorized execution.

### 6.1 Initialization and Solver Dispatch

The framework is initialized by defining the continuum mechanics properties alongside the specific AT2 phase-field parameters. To maximize modularity, users dynamically toggle the tension-compression energy splits and degradation functions via initialization flags. A residual stiffness stabilization parameter ( $\kappa = 10^{-8}$ ) is enforced universally during initialization to prevent singular matrices in fully damaged domains ( $\phi \rightarrow 1$ ).

Rather than hard-coding the iterative solver mathematics within the main script, `main.m` acts as a lightweight dispatcher for the adaptive load-stepping heuristic. At each increment, it updates the Dirichlet boundary conditions via the `updateDisplacementBC` subroutine. This rigidly partitions the degrees of freedom ( $\partial\Omega_D$ ) into static supports (locked to 0.0) and active loading surfaces (scaled dynamically by  $\lambda$ ).

The main script then invokes the `solveStaggeredLoadStep` routine, passing the previously accepted damage field as a strict thermodynamic lower bound ( $\phi_{\min}$ ). Based on the boolean convergence flags returned by the solvers, `main.m` autonomously accepts the trial state or rejects it, bisecting the increment ( $d\lambda$ ) and seamlessly reconstructing the reduced kinematic state from the last accepted thermodynamic variables.

### 6.2 High-Performance Mechanical Assembly

Within the staggered loop, the active displacement solve dictates that the global tangent stiffness matrix  $\mathbf{K}_u$  and the internal residual force vector  $\mathbf{R}_u$  must be reassembled at every Newton iteration. Any computational inefficiencies here will severely bottleneck the framework.

PFCOMPAS-v1.0.0 mitigates this by utilizing the `assembleDisplacementSystem` subroutine, which deploys a high-performance map-reduce architecture. Pre-allocated dense arrays act as thread-safe receptacles. A `parfor` loop distributes the complex mechanical integrations encompassing the evaluation of elastic constitutive tensors, spectral decompositions, and degradation coupling across available CPU cores via the

`elementDisplacementContribution` function. The local elemental stiffness matrices ( $\mathbf{K}_e$ ) and force vectors ( $\mathbf{R}_e$ ) are flattened and stored strictly into their designated column slices, absolutely precluding memory race conditions.

Following the parallel integrations, the global tangent stiffness matrix is instantiated via MATLAB's native `sparse` constructor, leveraging the 1D spatial indices precomputed during mesh initialization. To assemble the 1D global residual vector, the solver utilizes MATLAB's `accumarray` function:

```
R = accumarray(assemblyPattern.u.R, V_R(:), [ndof, 1], @sum, 0);
```

By mathematically mapping overlapping element boundaries and automatically accumulating their nodal force contributions, `accumarray` eliminates sequential `for` loops. This results in a purely vectorized assembly architecture that computes the global out-of-balance forces with near-compiled speeds.

### 6.3 Thread-Safe History Field Extraction

Between the displacement and phase-field Newton solves, the framework must update the history field  $\mathcal{H}$  to enforce thermodynamic irreversibility. To prevent the non-physical numerical smearing that occurs with nodal  $\mathcal{L}_2$  projections, `updateHistoryField` strictly maintains the history variables exclusively at the discrete Gauss integration points.

While the core mechanical assembly routine evaluates identical constitutive equations, reutilizing that specific function for the history update would be exceptionally computationally wasteful due to the overhead of constructing local 4th-order tangent stiffness matrices. To guarantee maximum execution speed, PFCOMPAS-v1.0.0 isolates the energy evaluation into a lightweight subroutine: `elementPositiveEnergy`.

During the parallel `parfor` history update loop, this function utilizes the converged nodal displacement vector to compute the local strain state, routes it through the `computeSplitInfo` function, explicitly extracts the scalar active tensile energy component ( $\psi_e^+$ ), and discards all auxiliary stress tensors. By bypassing all matrix assemblies, this localized subroutine evaluates the energetic driving force with near-optimal floating-point efficiency.

### 6.4 Vectorized Phase-Field Assembly and Active-Set Projection

The scalar phase-field evolution equation is resolved via the `solvePhaseFieldNewton` subroutine. Mirroring the mechanical solver, the global phase-field arrays ( $\mathbf{K}_\phi$  and  $\mathbf{R}_\phi$ ) are assembled using an identical `parfor` map-reduce architecture via `elementPhaseFieldContribution`, which explicitly evaluates the analytical derivatives of the degradation function ( $g'$  and  $g''$ ) to construct the exact Jacobian.

Crucially, PFCOMPAS-v1.0.0 handles the irreversibility constraint ( $\phi \geq \phi_{\min}$ ) directly within the linear algebra via an Active-Set projection, avoiding computationally expensive exterior penalty methods. The algorithm dynamically identifies nodes resting at their thermodynamic lower bound that are attempting to heal ( $R_i > 0$ ). The projected residual for this active set ( $\mathcal{A}$ ) is forced exactly to zero.

The solver then isolates the inactive set ( $\mathcal{I}$ ) nodes legally allowed to accumulate damage and solves a condensed linear sub-system using logical indexing:

```
dPhi(inactiveDOFs) = -K_phi(inactiveDOFs, inactiveDOFs) \ R_proj(inactiveDOF
```

Active DOFs receive a strict zero increment. By geometrically locking the healing nodes and reducing the dimensionality of the linear system, this strategy guarantees hard enforcement of crack irreversibility while accelerating the direct linear solver.

## 6.5 Global Energy Post-Processing

Explicitly integrating the global potential energy functional (II) at every sub-iteration is computationally prohibitive. Therefore, PFCOMPAS-v1.0.0 isolates the global energy evaluation into the `postprocessAT2Energy` subroutine, invoked exclusively after true global equilibrium is achieved for the accepted load step.

A `parfor` loop distributes the localized element integrations (`elementEnergy`) across the CPU workers. The degraded elastic energy and the regularized crack surface energy are numerically integrated via Gauss quadrature, leveraging the algebraic dot product of the gradient vector (`gradPhi' * gradPhi`) for the fracture energy term. These local scalar contributions are reduced via vectorized summation to yield the global network energies, which are then checked against the incremental external work ( $\Delta W_{\text{ext}}$ ) to explicitly print the thermodynamic energy balance to the console.

## 6.6 Boundary Traction Integration and Reaction Forces

To construct macroscopic load-displacement response curves, the `computeReactionForce` subroutine explicitly evaluates the contour integrals of the degraded Cauchy stress tensor  $\boldsymbol{\sigma}$  over the Dirichlet boundaries, avoiding the numerical noise associated with extracting nodal forces directly from the global residual vector.

This introduces a dimensional mismatch: Gauss-Legendre quadrature points  $s_q$  are generated in a 1D reference domain, but evaluating the 2D shape functions necessitates 2D parent coordinates  $(\xi, \eta)$ . The framework utilizes the `refEdgeCoords` mapping subroutine to dynamically project the 1D integration point directly onto the perimeter of the standard Q4 bi-unit reference element based on the local edge index.

To mathematically safeguard the thermodynamic orientation of the external work evaluation, the subroutine checks the outward normal vector  $\mathbf{n}$ . By cross-referencing against predefined Cartesian boundary labels ('top', 'bottom', etc.), the code dynamically reverses the orientation ( $\mathbf{n} \leftarrow -\mathbf{n}$ ) if an inward-pointing anomaly caused by arbitrary mesh winding is detected, ensuring the discrete reaction force is rigorously accumulated without sign errors.

# 7 A Posteriori Error Estimation and Diagnostics

Standard finite element codes rely heavily on tracking the normalized Newton-Raphson residual  $\mathbf{R}^{(k)}$  to determine convergence. However, algebraic convergence merely proves that the discrete system of equations  $\mathbf{K}\Delta\mathbf{u} = -\mathbf{R}$  has been solved correctly; it provides absolutely no information regarding the discretization error - the physical discrepancy between the exact analytical continuum solution and the discretized finite element approximation.

To rigorously gauge the accuracy of the spatial mesh size  $h$  and its interplay with the phase-field length scale  $l_0$ , the PFCOMPAS-v1.0.0 framework implements explicit *a posteriori* error estimators based on the formulations of Verfürth. These metrics autonomously

flag spatial regions requiring refinement by evaluating the localized violation of the continuous weak form equations.

## 7.1 Mechanical Displacement Error Estimator ( $\eta_u$ )

For the mechanical equilibrium equation  $\nabla \cdot (g(\phi)\boldsymbol{\sigma}^+ + \boldsymbol{\sigma}^-) = \mathbf{0}$ , the local error indicator for a given element  $K$  is defined by two bounded terms:

$$\eta_{u,K}^2 = h_K^2 \int_K \|\nabla \cdot \boldsymbol{\sigma}_{\text{deg}}\|^2 d\Omega + \sum_{E \subset \partial K} \frac{h_E}{2} \int_E \|[[\mathbf{t}]]\|^2 d\Gamma \quad (82)$$

where  $h_K$  is the characteristic element size (area),  $h_E$  is the edge length, and  $[[\mathbf{t}]] = [[\boldsymbol{\sigma}_{\text{deg}}\mathbf{n}]]$  denotes the jump in the traction vector across an element edge  $E$ .

### 7.1.1 Volumetric Residual Simplification

The first term evaluates the lack of equilibrium strictly inside the element volume. However, for low-order bilinear quadrilateral (Q4) elements, the second spatial derivatives of the displacement shape functions vanish. Consequently, without deploying advanced derivative projection or stress recovery algorithms (e.g., Zienkiewicz-Zhu patch recovery), the internal divergence  $\nabla \cdot \boldsymbol{\sigma}$  evaluates approximately to zero. The framework formally structures this calculation within the `u2DResidualIndicator` subroutine but mathematically bypasses the integration to focus computational resources on the dominant edge flux jumps.

### 7.1.2 Traction Jump Residuals and Dirichlet Masking

The dominant source of discretization error in low-order finite elements manifests as discontinuous stresses across element interfaces. Using the explicitly mapped topological database (`mesh.edge`), the subroutine systematically integrates the traction vector on both the “left” and “right” sides of every interior edge utilizing 1D Gauss quadrature. The magnitude of the discontinuity  $\mathbf{t}_L - \mathbf{t}_R$  provides a direct, highly localized proxy for the solution error.

Crucially, because error estimators evaluate the violation of the test function space ( $\delta\mathbf{u}$ ), they must identically vanish on Dirichlet boundaries where displacements are rigidly prescribed ( $\delta\mathbf{u} = \mathbf{0}$ ). The framework dynamically cross-references the mesh topology against the `dir_nodes` array, safely masking and skipping these boundaries to prevent the artificial inflation of the global error metric. The resulting scalar jump contribution is mathematically halved and distributed symmetrically to the adjacent left and right elements, ultimately yielding the elemental indicator  $\eta_{\text{elem},u}$  and the single global spatial error bound  $\eta_u$ .

## 7.2 Phase-Field Damage Error Estimator ( $\eta_\phi$ )

Parallel to the mechanical equilibrium evaluation, the framework measures the continuous spatial discretization error of the damage variable. The phase-field evolution PDE represents a complex balance between the thermodynamic driving force and the energetic resistance to fracture:

$$g'(\phi)\mathcal{H} + \frac{G_c}{l_0}\phi - G_cl_0\Delta\phi = 0 \quad (83)$$

The Verfürth *a posteriori* error indicator for the phase-field,  $\eta_\phi$ , is similarly constructed by bounding the violation of this strong form. For a given element  $K$ , the squared error is defined as:

$$\eta_{\phi,K}^2 = h_K^2 \int_K \left\| g'(\phi)\mathcal{H} + \frac{G_c}{l_0}\phi - G_cl_0\Delta\phi \right\|^2 d\Omega + \sum_{E \subset \partial K} \frac{h_E}{2} \int_E \|[[\mathbf{F}_\phi \cdot \mathbf{n}]]\|^2 d\Gamma \quad (84)$$

### 7.2.1 Internal Diffusion and Edge Flux Jumps

During the evaluation of the volumetric residual inside the `phi2DResidualIndicator` subroutine, the Laplacian of the damage field ( $\Delta\phi$ ) evaluates to zero due to the linear nature of the Q4 shape function derivatives. The remaining terms are the non-linear energetic driving force ( $g'(\phi)\mathcal{H}$ ) and the linear restoration force ( $(G_c/l_0)\phi$ ) which are strictly evaluated using the Gauss integration points to preserve the exact value of the discrete history field  $\mathcal{H}_q$ .

The second term evaluates the discontinuity of the gradient flux vector,  $\mathbf{F}_\phi = -G_cl_0\nabla\phi$ , across the element interfaces. Mirroring the mechanical indicator, the flux vectors on the left and right sides of each interior edge are projected onto the outward normal  $\mathbf{n}$ . The magnitude of the resulting scalar jump,  $\|[[\mathbf{F}_\phi \cdot \mathbf{n}]]\|$ , serves as a highly localized metric identifying regions where the phase-field spatial gradient is poorly resolved by the underlying finite element mesh.

### 7.2.2 Global Mixed Indicator ( $\eta_{\text{mix}}$ )

The element-wise errors are mapped back to their originating quadrilateral geometries and globally accumulated. To achieve a unified metric assessing the comprehensive accuracy of the fully coupled state, the independent mechanical and phase-field indicators are combined into an aggregated global mixed indicator via their  $\mathcal{L}_2$  norms:

$$\eta_{\text{mix}} = \sqrt{\eta_u^2 + \eta_\phi^2} \quad (85)$$

By continuously outputting this  $\eta_{\text{mix}}$  alongside the individual residual norms, the PFCOMPAS-v1.0.0 framework provides researchers with an autonomous, high-fidelity metric to definitively prove that algorithmic convergence guarantees rigorous compliance with the continuous governing PDEs of the Francfort-Marigo formulation.

## 8 High-Fidelity Visualization and VTK Data Export

Phase-field modeling generates an immense, continuously evolving landscape of multi-dimensional data. A standard 2D or 3D simulation tracks the macroscopic structural displacements ( $\mathbf{u}$ ), the highly localized scalar damage parameter ( $\phi$ ), the degraded continuum stress tensors ( $\boldsymbol{\sigma}$ ), and the element-wise spatial discretization error estimators ( $\eta$ ). Attempting to render these dynamic, high-density arrays natively within the MATLAB environment invokes severe graphical processing bottlenecks and frequently results in memory allocation crashes.

To ensure computational scalability and to provide users with publication-quality visualization capabilities, the PFCOMPAS-v1.0.0 framework rigorously delegates all post-processing rendering to ParaView, the open-source data analysis application. This pipeline is executed via the `writeVTUStep` subroutine, which autonomously exports the converged simulation states into the VTK XML Unstructured Grid (`.vtu`) format.

## 8.1 Timestep-Local Memory Architecture

In explicit dynamic simulations or simulations employing heavily refined adaptive load-stepping heuristics, resolving the complete crack propagation history may require thousands of individual load steps. Aggregating the fully coupled state arrays across every timestep into global MATLAB variables would inevitably exceed physical RAM capacities.

To guarantee that the framework operates with a strictly bounded memory footprint, the VTK export sequence is inherently “timestep-local.” At the conclusion of an accepted load step, the main execution script passes only the instantaneous state to `writeVTUStep`. The subroutine writes the `.vtu` file directly to the storage disk and immediately clears the arrays from active memory, allowing the solver to infinitely advance in pseudo-time without suffering performance degradation.

## 8.2 Data Segregation: Kinematics vs. Mechanics

The subroutine structurally parses the output data into two distinct VTK typologies, adhering to the mathematical nature of the finite element fields:

- **PointData (Nodal Fields):** Variables derived directly from the  $C^0$ -continuous finite element shape functions. This includes the macroscopic displacement vector field  $\mathbf{u}$  and the scalar phase-field damage variable  $\phi$ . By storing these as point data, ParaView can naturally apply continuous nodal interpolations for smooth contour rendering.
- **CellData (Elemental Fields):** Variables derived from the spatial gradients of the shape functions ( $\nabla \mathbf{N}$ ), which are inherently discontinuous across element boundaries in standard low-order formulations. This includes the averaged Cauchy stress tensor components ( $\sigma_{xx}, \sigma_{yy}, \sigma_{xy}$ ) and the Verfürth estimators ( $\eta_u, \eta_\phi, \eta_{\text{mix}}$ ). Storing these as piecewise constant cell data accurately reflects the discrete nature of the element integrations and flux jumps.

## 8.3 C++ Indexing Parity

A critical software engineering bridge within the export sequence is the translation of array indexing conventions. MATLAB fundamentally utilizes 1-based indexing for its arrays. However, the VTK standard, rooted deeply in C and C++ architectures, strictly mandates 0-based indexing for topological mappings. The framework explicitly enforces the transformation `conn0 = conn - 1` prior to writing the element connectivity blocks, preemptively eliminating parser corruptions when the unstructured grid is loaded into ParaView’s C++ rendering engine.

### 8.3.1 Topological Translation and VTK Cell Dictionaries

While the internal PFCOMPAS-v1.0.0 framework utilizes standard finite element nomenclature to define spatial discretizations (e.g., Q4 for 4-node quadrilaterals, H8 for 8-node hexahedra), the VTK Unstructured Grid standard relies on a strictly enumerated C++ dictionary of integer identifiers to render element topologies. To bridge this semantic gap, the `vtkCellTypeFromElemType` subroutine acts as an automated translation layer. By

evaluating the string identifier stored in the `mesh.meta.elemType` struct, the function maps the geometry to the corresponding VTK constant:

- **Triangles (T3):** Mapped to VTK\_TRIANGLE (ID 5).
- **Quadrilaterals (Q4):** Mapped to VTK\_QUAD (ID 9).
- **Tetrahedra (tet4):** Mapped to VTK\_TETRA (ID 10).
- **Hexahedra (H8):** Mapped to VTK\_HEXAHEDRON (ID 12).

This integer array is subsequently written to the `<types>` DataArray block within the `.vtu` file. By explicitly managing this dictionary translation at the point of export, the framework maintains the flexibility to define custom, application-specific element nomenclature internally.

## 8.4 Dynamic Field Parsing and Array Validation

Because phase-field fracture mechanics is an inherently multiphysics problem, the exported datasets contain a heterogeneous mix of scalar fields (e.g.,  $\phi$ ,  $\mathcal{H}$ ,  $\eta$ ) alongside multidimensional vector and tensor fields (e.g.,  $\mathbf{u}$ ,  $\boldsymbol{\sigma}$ ). To maintain an extensible software architecture, the `writeVTUFieldData` subroutine acts as a dynamic structure parser. It autonomously iterates through the generic MATLAB struct, dynamically extracting the field names to label the VTK `<DataArray>` blocks and querying the array dimensions to populate the `NumberOfComponents` XML attribute (differentiating scalars from tensors on the fly).

## 8.5 Temporal Collections and Pseudo-Time Tracking (.pvd)

The simulation of brittle fracture using the AT2 phase-field model is intrinsically path-dependent. Consequently, an adaptive simulation generates a vast chronological series of individual `.vtu` unstructured grid files. Manually importing hundreds of discrete files into a visualization engine is highly inefficient and strips the datasets of their critical physical context (e.g., the applied displacement  $\lambda$ ). To elegantly preserve this temporal mapping, the PFCOMPAS-v1.0.0 framework constructs a ParaView Data (`.pvd`) collection file.

The `.pvd` file operates as a master XML wrapper. The `initializePVDCollection` subroutine establishes the structural header of this file at the onset of the simulation:

```
<VTKFile type="Collection" version="0.1" byte_order="LittleEndian">
  <Collection>
```

### 8.5.1 HPC File I/O Safety Protocol

A critical software engineering practice implemented within this routine is the immediate closure of the file stream (`fclose(fid)`) after the header is generated. In large-scale computational mechanics, maintaining an open I/O file stream across the entire duration of the solve renders the dataset highly vulnerable to catastrophic corruption in the event of a cluster crash, power failure, or node timeout. By utilizing an append-based architecture, the framework guarantees that the visualization sequence remains fully recoverable and uncorrupted, even if the primary MATLAB execution halts unexpectedly.

### 8.5.2 Strict Topological Validation

The ParaView VTK rendering engine is highly susceptible to silent failures if an appended data array does not perfectly match the declared spatial geometry. To absolutely preclude this vulnerability, the subroutine enforces a strict pre-write validation check. Before a single byte of data is streamed to the hard drive, the algorithm evaluates the row count of every numerical field against the **nEntities** parameter ( $n_{\text{nodes}}$  for **PointData** and  $n_{\text{elements}}$  for **CellData**). If a dimensional discrepancy is detected, the solver halts immediately. By printing the numeric data utilizing high-precision 16g ASCII format, the framework additionally guarantees that infinitesimal variations in the damage variable are perfectly preserved.

### 8.5.3 Dynamic Timestep Appending and Dataset Portability

As the phase-field solver sequentially writes individual **.vtu** files, the **appendPVDEntry** subroutine manages the chronological indexing. It opens the **.pvd** file in append mode ('a') and injects a new **<DataSet>** element:

```
<DataSet timestep="0.005" group="" part="0" file="SENT_0042.vtu"/>
```

The **timestep** attribute explicitly binds the instantaneous 3D spatial field to a 1D continuous scalar parameter. This allows the ParaView engine to accurately plot the non-linear temporal gaps between frames, which is particularly vital during catastrophic failure where the adaptive solver violently reduces the increment size.

Before appending the XML entry, the framework strips the absolute directory path from the filename string utilizing MATLAB's **fileparts** function. By indexing strictly the relative filename, the output folder is rendered entirely self-contained, ensuring seamless cross-platform visualization when transferring data packets from Linux HPC clusters to local workstations.

### 8.5.4 XML Finalization and Pipeline Closure

Upon termination of the simulation, the master **.pvd** collection must be structurally sealed to satisfy strict XML parsing standards. The **finalizePVDCollection** subroutine is invoked to reopen the index file one final time and append the closing XML block:

```
</Collection>  
</VTKFile>
```

By formally closing these tags, the framework prevents the ParaView rendering engine from throwing syntax parsing errors upon initialization, ensuring immediate access to the complete chronological animation of the fracturing domain.